

1. Project Overview

Project Topic: Singapore Hawker Centres' Temporary Closure ETL & Analysis

This project investigates the patterns, frequency, and regional distribution of temporary hawker centre closures across Singapore, using an automated ETL pipeline to collect, clean and analyse closure schedules. Using geospatial and temporal data, we examine whether closure timings are equitably distributed across Singapore and how these temporary shutdowns may affect community food accessibility. Our findings aim to support data-driven insights and policy recommendations that help balance hygiene maintenance with residents' access to affordable local food options.

Our brainstormed topics before the above was decided upon are as follows:

Brainstormed Topics
• Heat Effect in Singapore
• Air Quality in Singapore (How air quality differs between different parts of Singapore)
• Relationship between aircon usage and heat
• Hawker centre temporary closure schedules
• Smart City "Livability" Index (Which neighbourhood in Singapore is the best place to live based on data)

Problem statements:

- Investigate if hawker centres' temporary closure schedules are equitably distributed across Singapore
- Assess the impact of hawker centre temporary closures on food accessibility and propose policy recommendations

2. Data Sources

Our data sources for this project are as follows:

2.1

https://data.gov.sg/datasets/d_bda4baa634dd1cc7a6c7cad5f19e2d68/view

Dates of Hawker Centre Closures; data by National Environmental Agency (NEA)

2.2

We also downloaded the CSV file available from the link above and analysed the data before drawing out the Extract, Transforming and Loading (ETL) process. This csv file titled "hawker_closures" is included in our "Project Artefacts" Folder.

2.3

We also ran the API call script provided in 2.1 in Jupyter Notebook. API call script is as below:

Python ▾

```
import requests

dataset_id = "d_bda4baa634dd1cc7a6c7cad5f19e2d68"
url = "https://data.gov.sg/api/action/datastore_search?resource_id=" + dataset_id

response = requests.get(url)
print(response.json())
```

3. Methodology / Approach

This project adopts a structured, database-driven methodology to analyse hawker centre closures in Singapore. PostgreSQL is used as the core data management platform in this project. Our approach focuses on building a clean, reliable, and reproducible data pipeline that transforms raw closure information into an analysis-ready dataset.

3.1 Technology Stack

The project uses a lightweight but robust stack centred on:

- **PostgreSQL** — primary database for storage, cleaning, validation, and analysis
- **pgAdmin / SQL Shell** — for database administration and query execution
- **SQL** — for schema creation, data preparation, transformation, and analytical queries

This stack ensures consistency, transparency, and full control over the data lifecycle.

3.2 ETL Workflow

The ETL process is designed to systematically convert raw closure data into structured tables suitable for analysis.

Raw hawker centre closure data is sourced from publicly available datasets as mentioned in Section 2. We downloaded data in csv format for referencing purposes as we wanted to ensure that our API call script returns the same data as the csv file.

3.2.1 Extraction process

- Import source code

```
# 1 Imports

import pandas as pd
from sqlalchemy import create_engine, text
```

- Configuration source code

```
# 2 Configuration

# PostgreSQL connection string
DB_URL = "postgresql://postgres:bingka@localhost:5432/hawkerdb"

engine = create_engine(DB_URL)
```

- Helper functions source code

```
# 3 Helper Functions

def assign_region(lat, lon):
    """
    Rough region classification based on coordinates.
    """
    if pd.isna(lat) or pd.isna(lon):
        return None

    if lat > 1.40:
        return "North"
    if lon > 103.90:
        return "East"
    if lat < 1.32:
        return "South"
    if lon < 103.75:
        return "West"
    return "Central"
```

3.2.2 Purpose of Helper Function

This function is used to **assign a geographic region** (North, East, South, West, Central) to each hawker centre based on its latitude and longitude coordinates.

It supports our **data transformation** step by enriching the dataset with a region field that is not provided directly by the API. Since the NEA API does not include a region field, this helper function enriches the dataset by assigning each hawker centre to a region based on its coordinates. This enables regional analysis in the later stages of the project.

The function checks the coordinates against predefined thresholds and returns one of five region labels: North, East, South, West, or Central.

The rules of the function are as follows:

- If latitude is **greater than 1.40** → classify as **North**
- If longitude is **greater than 103.90** → classify as **East**
- If latitude is **less than 1.32** → classify as **South**
- If longitude is **less than 103.75** → classify as **West**
- Otherwise → classify as **Central**

Note:

These thresholds are approximate and used to create a general regional grouping for analysis purposes.

3.2.3 Extraction source codes

- Extract (Load from API) source code

```
# 4 Extract (Load from API)

dataset_id = "d_bda4baa634dd1cc7a6c7cad5f19e2d68"
base_url = "https://data.gov.sg/api/action/datastore_search"

all_records = []
limit = 100
offset = 0

while True:
    url = f"{base_url}?resource_id={dataset_id}&limit={limit}&offset={offset}"
    response = requests.get(url).json()

    records = response["result"]["records"]
    all_records.extend(records)

    # Stop when fewer than 100 rows returned
    if len(records) < limit:
        break

    offset += limit

df = pd.DataFrame(all_records)
df.head()
```

First attempt of extraction returned this error message:

```
-----  
NameError                                Traceback (most recent call last)  
Cell In[7], line 12  
    10 while True:  
    11     url = f"{base_url}?resource_id={dataset_id}&limit={limit}&offset={offset}"  
--> 12     response = requests.get(url).json()  
    14     records = response["result"]["records"]  
    15     all_records.extend(records)  
  
NameError: name 'requests' is not defined  
  
Fix Code
```

When this error happens it means that the “requests” library was not imported before usage.

Thus the import source code was amended from this;

```
# 1 Imports  
  
import pandas as pd  
from sqlalchemy import create_engine, text
```

To this:

```
•[2]: # 1 Imports  
  
import pandas as pd  
import requests  
from sqlalchemy import create_engine, text
```

The statement “import requests” was added to source code. Once this was amended the following output was generated.

[11]:

	_id	serial_no	name	q1_cleaningstartdate	q1_cleaningenddate	remarks_q1	q2_cleaningstartdate	q2_cl
0	1	1	Adam Road Food Centre	30/3/2026	31/3/2026	nil	8/6/2026	
1	2	2	Aljunied Ave 2 Blk 117 (Blk 117 Aljunied Marke...	9/3/2026	10/3/2026	nil	8/6/2026	
2	3	3	Amoy Street Food Centre (Telok Ayer Food Centre)	28/2/2026	28/2/2026	nil	30/5/2026	
3	4	120	Anchorvale Village Hawker Centre	16/3/2026	17/3/2026	nil	15/6/2026	
4	5	4	Ang Mo Kio Ave 1 Blk 226D (Kebun Baru Market a...	9/3/2026	11/3/2026	nil	1/6/2026	

5 rows × 28 columns

3.2.4 Pagination Logic

Before running the Extract (Load from API) source code above, we also performed a test to ensure that the API call will return all 123 rows as reflected in the “hawker_closures” csv file. This is to ensure that data generated is accurate for analysis purposes. The following source code was ran. However only 100 rows were returned. The NEA Datastore API only returns **up to 100 records per request**. The source code before pagination is as follows:

Extraction source code before pagination:

4 Extract (Load from API instead of CSV)

```
import requests
import pandas as pd

dataset_id = "d_bda4baa634dd1cc7a6c7cad5f19e2d68"
url = "https://data.gov.sg/api/action/datastore_search?resource_id=" + dataset_id

response = requests.get(url)
data = response.json()

df = pd.DataFrame(data["result"]["records"])

df.head()
```

Tests ran to check output:

```
url = "https://data.gov.sg/api/action/datastore_search?resource_id=d_bda4baa634dd1cc7a6c7cad5f19e2d68"
response = requests.get(url).json()
print(response["result"].keys())
```

```
-----
KeyError                                Traceback (most recent call last)
Cell In[7], line 3
      1 url = "https://data.gov.sg/api/action/datastore_search?resource_id=d_bda4baa634dd1cc7a6c7cad5f19e2d68&limit=100"
      2 response = requests.get(url).json()
----> 3 print(response["result"].keys())

KeyError: 'result'
```

```
url = "https://data.gov.sg/api/action/datastore_search?resource_id=d_bda4baa634dd1cc7a6c7cad5f19e2d68"
response = requests.get(url).json()
print(response["result"].keys())
```

```
dict_keys(['resource_id', 'fields', 'records', '_links', 'total', 'limit'])
```

```
print(response["result"]["limit"])
```

100

```

import requests

dataset_id = "d_bda4baa634dd1cc7a6c7cad5f19e2d68"
url = f"https://data.gov.sg/api/action/datastore_search?resource_id={dataset_id}"

response = requests.get(url).json()
result = response["result"]

print("Keys in result:", list(result.keys()))
print("Rows returned in this call:", len(result["records"]))
print("Total rows in dataset:", result["total"])

if len(result["records"]) < result["total"]:
    print("⚠ Not all rows returned – pagination is required.")
else:
    print("✅ All rows returned – no pagination needed.")

Keys in result: ['resource_id', 'fields', 'records', '_links', 'total']
Rows returned in this call: 100
Total rows in dataset: 123
⚠ Not all rows returned – pagination is required.

```

Thus we had to amend the Extraction source code to include pagination. Amended extraction code is as follows:

```

# 4 Extract (Load from API)

dataset_id = "d_bda4baa634dd1cc7a6c7cad5f19e2d68"
base_url = "https://data.gov.sg/api/action/datastore_search"

all_records = []
limit = 100
offset = 0

while True:
    url = f"{base_url}?resource_id={dataset_id}&limit={limit}&offset={offset}"
    response = requests.get(url).json()

    records = response["result"]["records"]
    all_records.extend(records)

    # Stop when fewer than 100 rows returned
    if len(records) < limit:
        break

    offset += limit

df = pd.DataFrame(all_records)
df.head()

```

3.3 Putting data into the tables:

The following source code was used:

```
# 5 Transform: Hawker Centres Table

hawker_cols = {
    "name": "name",
    "address_myenv": "address",
    "latitude_hc": "latitude",
    "longitude_hc": "longitude",
    "no_of_food_stalls": "num_food_stalls",
    "no_of_market_stalls": "num_market_stalls"
}

hawkers = df[list(hawker_cols.keys())].rename(columns=hawker_cols)

# Assign region
hawkers["region"] = hawkers.apply(
    lambda r: assign_region(r["latitude"], r["longitude"]),
    axis=1
)

# Remove duplicates
hawkers = hawkers.drop_duplicates(subset=["name"]).reset_index(drop=True)

hawkers.head()
```

However, it returned the following errors:

```
TypeError                                Traceback (most recent call last)
Cell In[13], line 15
    12 hawkers = df[list(hawker_cols.keys())].rename(columns=hawker_cols)
    14 # Assign region
--> 15 hawkers["region"] = hawkers.apply(
    16     lambda r: assign_region(r["latitude"], r["longitude"]),
    17     axis=1
    18 )
    20 # Remove duplicates
    21 hawkers = hawkers.drop_duplicates(subset=["name"]).reset_index(drop=True)

File ~\anaconda3\Lib\site-packages\pandas\core\frame.py:10401, in DataFrame.apply(self, func, axis, raw, result_type, args, by_row, engine, engine_kwargs, **kwargs)
    10387 from pandas.core.apply import frame_apply
    10389 op = frame_apply(
    10390     self,
    10391     func=func,
    (... ) 10399     kwargs=kwargs,
    10400 )
> 10401 return op.apply().__finalize__(self, method="apply")

File ~\anaconda3\Lib\site-packages\pandas\core\apply.py:916, in FrameApply.apply(self)
    913 elif self.raw:
    914     return self.apply_raw(engine=self.engine, engine_kwargs=self.engine_kwargs)
--> 916 return self.apply_standard()
```

```

File ~\anaconda3\Lib\site-packages\pandas\core\apply.py:1063, in FrameApply.apply_standard(self)
    1061 def apply_standard(self):
    1062     if self.engine == "python":
-> 1063         results, res_index = self.apply_series_generator()
    1064     else:
    1065         results, res_index = self.apply_series_numba()

File ~\anaconda3\Lib\site-packages\pandas\core\apply.py:1081, in FrameApply.apply_series_generator(self)
    1078 with option_context("mode.chained_assignment", None):
    1079     for i, v in enumerate(series_gen):
    1080         # ignore SettingWithCopy here in case the user mutates
-> 1081         results[i] = self.func(v, *self.args, **self.kwargs)
    1082         if isinstance(results[i], ABCSeries):
    1083             # If we have a view on v, we need to make a copy because
    1084             # series_generator will swap out the underlying data
    1085             results[i] = results[i].copy(deep=False)

Cell In[13], line 16, in <lambda>(r)
    12 hawkers = df[list(hawker_cols.keys())].rename(columns=hawker_cols)
    14 # Assign region
    15 hawkers["region"] = hawkers.apply(
--> 16     lambda r: assign_region(r["latitude"], r["longitude"]),
    17     axis=1
    18 )
    20 # Remove duplicates
    21 hawkers = hawkers.drop_duplicates(subset=["name"]).reset_index(drop=True)

Cell In[10], line 10, in assign_region(lat, lon)
     7 if pd.isna(lat) or pd.isna(lon):
     8     return None
--> 10 if lat > 1.40:
    11     return "North"
    12 if lon > 103.90:

TypeError: '>' not supported between instances of 'str' and 'float'

```

After analysing this error;

```

TypeError: '>' not supported between instances of 'str' and 'float'

```

it was found that this error happened because the “latitude” and “longitude” values are strings not numbers. The “assign_region()” function compares them to floats ie “if lat > 1.40:” thus Python is unable to compare the “1.3521” (string) with 1.40 (float), so it crashes. Thus before applying “assign_region()”, the coordinate columns must be converted to numeric. The source code was then amended to the following:

```
# 5.1 Transform: Hawker Centres Table: amended source code

hawker_cols = {
    "name": "name",
    "address_myenv": "address",
    "latitude_hc": "latitude",
    "longitude_hc": "longitude",
    "no_of_food_stalls": "num_food_stalls",
    "no_of_market_stalls": "num_market_stalls"
}

hawkers = df[list(hawker_cols.keys())].rename(columns=hawker_cols)

# Convert coordinates to numeric
hawkers["latitude"] = pd.to_numeric(hawkers["latitude"], errors="coerce")
hawkers["longitude"] = pd.to_numeric(hawkers["longitude"], errors="coerce")

# Assign region
hawkers["region"] = hawkers.apply(
    lambda r: assign_region(r["latitude"], r["longitude"]),
    axis=1
)

# Remove duplicates
hawkers = hawkers.drop_duplicates(subset=["name"]).reset_index(drop=True)

hawkers.head()
```

And the output is as follows:

	name	address	latitude	longitude	num_food_stalls	num_market_stalls	region
0	Adam Road Food Centre	2, Adam Road, Singapore 289876	1.324132	103.814163	32	0	Central
1	Aljunied Ave 2 Blk 117 (Blk 117 Aljunied Marke...	Blk 117, Aljunied Ave 2, Singapore 380117	1.320710	103.887093	79	82	Central
2	Amoy Street Food Centre (Telok Ayer Food Centre)	National Development Building, Annex B, Telok ...	1.279129	103.846985	134	1	South
3	Anchorvale Village Hawker Centre	339 Anchorvale Road, Singapore 540339	1.396793	103.888437	36	0	Central
4	Ang Mo Kio Ave 1 Blk 226D (Kebun Baru Market a...	Blk 226D, Ang Mo Kio Ave 1, Singapore 564226	1.366950	103.839188	10	101	Central

4. Transformation / Data Preparation

Before loading, the data underwent cleaning and standardization which includes:

- Normalising date formats
- Standardising hawker centre names
- Handling missing or inconsistent values
- Ensuring region labels follow a consistent classification

These steps ensure that the data set is uniform and suitable for relational modelling.

Source code for inserting data:

```
# 6 Load: Insert Hawker Centres into PostgreSQL

with engine.begin() as conn:
    hawkers.to_sql("hawker_centres", conn, if_exists="append", index=False)

# Build mapping name → hawker_id
with engine.begin() as conn:
    hc_map = pd.read_sql(text("SELECT hawker_id, name FROM hawker_centres"), conn)

name_to_id = dict(zip(hc_map["name"], hc_map["hawker_id"]))
```

Source code for transforming data:

```
# 7 Transform: Closure Events Table
```

```
closure_rows = []

for _, row in df.iterrows():
    name = row["Name"]

    # Q1-Q4 cleaning closures
    for q in ["Q1", "Q2", "Q3", "Q4"]:
        start_col = f"{q} Cleaningstartdate"
        end_col = f"{q} Cleaningenddate"
        remarks_col = f"Remarks {q}"

        start = row.get(start_col)
        end = row.get(end_col)

        if pd.notna(start) and pd.notna(end):
            closure_rows.append({
                "name": name,
                "closure_type": "Cleaning",
                "start_date": pd.to_datetime(start, dayfirst=True).date(),
                "end_date": pd.to_datetime(end, dayfirst=True).date(),
                "remarks": row.get(remarks_col)
            })
```

```
    # Other works
    other_start = row.get("Other Works Startdate")
    other_end = row.get("Other Works Enddate")
    other_remarks = row.get("Remarks Other Works")

    if pd.notna(other_start) and pd.notna(other_end):
        closure_rows.append({
            "name": name,
            "closure_type": "Other Works",
            "start_date": pd.to_datetime(other_start, dayfirst=True).date(),
            "end_date": pd.to_datetime(other_end, dayfirst=True).date(),
            "remarks": other_remarks
        })

closures = pd.DataFrame(closure_rows)

# Map hawker names to IDs
closures["hawker_id"] = closures["name"].map(name_to_id)

# Keep only needed columns
closures = closures[["hawker_id", "closure_type", "start_date", "end_date", "remarks"]]

# Drop rows where hawker_id could not be mapped
closures = closures.dropna(subset=["hawker_id"])
closures["hawker_id"] = closures["hawker_id"].astype(int)

closures.head()
```

The transforming data source code generated the following errors:

```
-----  
KeyError                                Traceback (most recent call last)  
File ~\anaconda3\Lib\site-packages\pandas\core\indexes\base.py:3812, in Index.get_loc(self, key)  
    3811 try:  
-> 3812     return self._engine.get_loc(casted_key)  
    3813 except KeyError as err:  
  
File pandas/_libs/index.pyx:167, in pandas._libs.index.IndexEngine.get_loc()  
  
File pandas/_libs/index.pyx:196, in pandas._libs.index.IndexEngine.get_loc()  
  
File pandas/_libs/hashtable_class_helper.pxi:7088, in pandas._libs.hashtable.PyObjectHashTable.get_item()  
  
File pandas/_libs/hashtable_class_helper.pxi:7096, in pandas._libs.hashtable.PyObjectHashTable.get_item()  
  
KeyError: 'Name'
```

The above exception was the direct cause of the following exception:

```
KeyError                                Traceback (most recent call last)  
Cell In[16], line 6  
      3 closure_rows = []  
      5 for _, row in df.iterrows():  
----> 6     name = row["Name"]  
      8     # Q1-Q4 cleaning closures  
      9     for q in ["Q1", "Q2", "Q3", "Q4"]:
```

```

File ~\anaconda3\Lib\site-packages\pandas\core\series.py:1133, in Series.__getitem__(self, key)
    1130     return self._values[key]
    1132 elif key_is_scalar:
-> 1133     return self._get_value(key)
    1135 # Convert generator to list before going through hashable part
    1136 # (We will iterate through the generator there to check for slices)
    1137 if is_iterator(key):

File ~\anaconda3\Lib\site-packages\pandas\core\series.py:1249, in Series._get_value(self, label, take
able)
    1246     return self._values[label]
    1248 # Similar to Index.get_value, but we do not fall back to positional
-> 1249 loc = self.index.get_loc(label)
    1251 if is_integer(loc):
    1252     return self._values[loc]

File ~\anaconda3\Lib\site-packages\pandas\core\indexes\base.py:3819, in Index.get_loc(self, key)
    3814     if isinstance(casted_key, slice) or (
    3815         isinstance(casted_key, abc.Iterable)
    3816         and any(isinstance(x, slice) for x in casted_key)
    3817     ):
    3818         raise InvalidIndexError(key)
-> 3819     raise KeyError(key) from err
    3820 except TypeError:
    3821     # If we have a listlike key, _check_indexing_error will raise
    3822     # InvalidIndexError. Otherwise we fall through and re-raise
    3823     # the TypeError.
    3824     self._check_indexing_error(key)

KeyError: 'Name'

```

[Fix Code](#)

It was found that the errors above were generated as the "Q1 Cleaningstartdate", "Q1 Cleaningenddate", "Remarks Q1", "Other Works Startdate" do not exist.

Thus the code was amended to:

```
# 7.2 Finalised corrected code for Transform: Closure Events Table
```

```
closure_rows = []

for _, row in df.iterrows():
    name = row["name"]

    # Q1-Q4 cleaning closures
    for q in ["q1", "q2", "q3", "q4"]:
        start_col = f"{q}_cleaningstartdate"
        end_col = f"{q}_cleaningenddate"
        remarks_col = f"remarks_{q}"

        start = row.get(start_col)
        end = row.get(end_col)

        # Convert safely
        start_dt = pd.to_datetime(start, dayfirst=True, errors="coerce")
        end_dt = pd.to_datetime(end, dayfirst=True, errors="coerce")

        if pd.notna(start_dt) and pd.notna(end_dt):
            closure_rows.append({
                "name": name,
                "closure_type": "Cleaning",
                "start_date": start_dt.date(),
                "end_date": end_dt.date(),
                "remarks": row.get(remarks_col)
            })

    # Other works
    other_start = pd.to_datetime(row.get("other_works_startdate"), dayfirst=True, errors="coerce")
    other_end = pd.to_datetime(row.get("other_works_enddate"), dayfirst=True, errors="coerce")
    other_remarks = row.get("remarks_other_works")

    if pd.notna(other_start) and pd.notna(other_end):
        closure_rows.append({
            "name": name,
            "closure_type": "Other Works",
            "start_date": other_start.date(),
            "end_date": other_end.date(),
            "remarks": other_remarks
        })

# Convert to DataFrame
closures = pd.DataFrame(closure_rows)

# Map hawker names to IDs
closures["hawker_id"] = closures["name"].map(name_to_id)

# Keep only needed columns
closures = closures[["hawker_id", "closure_type", "start_date", "end_date", "remarks"]]

# Drop rows where hawker_id could not be mapped
closures = closures.dropna(subset=["hawker_id"])
closures["hawker_id"] = closures["hawker_id"].astype(int)

closures.head()
```


The following output was generated with this corrected source code:

	hawker_id	closure_type	start_date	end_date	remarks
0	1	Cleaning	2026-03-30	2026-03-31	nil
1	1	Cleaning	2026-06-08	2026-06-08	nil
2	1	Cleaning	2026-09-07	2026-09-07	nil
3	1	Cleaning	2026-12-07	2026-12-08	nil
4	2	Cleaning	2026-03-09	2026-03-10	nil

5. Loading

Cleaned data is inserted into PostgreSQL using SQL INSERT statements. The loading process is structured to maintain referential integrity and avoid duplication.

Loading source code:

```
# 8 Load: Insert Closure Events into PostgreSQL

with engine.begin() as conn:
    closures.to_sql("closure_events", conn, if_exists="append", index=False)

print("ETL completed successfully.")
```

Output:

```
ETL completed successfully.
```

Here are the table screenshots of the “hawkerdb” database from PostGreSQL.

“closure_events” table:

Dependents X Processes X hawkerdb/postgre... X public.closure_events/hawkerdb/postgres@PostgreSQL 18 X

public.closure_events/hawkerdb/postgres@PostgreSQL 18

Query Query History

```

1 SELECT * FROM public.closure_events
2 ORDER BY closure_id ASC

```

Data Output Messages Notifications

Showing rows: 1 to 515 Page No: 1

	closure_id [PK] integer	hawker_id integer	closure_type text	start_date date	end_date date	remarks text
145	145	34	Cleaning	2026-12-21	2026-12-21	nil
146	146	34	Other Works	2024-11-18	2024-12-17	Repairs and Redecoration
147	147	35	Cleaning	2026-01-05	2026-01-07	nil
148	148	35	Cleaning	2026-04-06	2026-04-08	nil
149	149	35	Cleaning	2026-07-06	2026-07-08	nil
150	150	35	Cleaning	2026-10-05	2026-10-09	nil
151	151	36	Cleaning	2026-03-30	2026-03-30	nil
152	152	36	Cleaning	2026-06-22	2026-06-24	nil
153	153	36	Cleaning	2026-09-28	2026-09-28	nil
Total rows: 515 Query complete 00:00:00.184						

“hawker_centres” table:

Dependents X Processes X hawkerdb/postgre... X public.hawker_centres/hawkerdb/postgres@PostgreSQL 18 X

public.hawker_centres/hawkerdb/postgres@PostgreSQL 18

Query Query History

```

1 SELECT * FROM public.hawker_centres
2 ORDER BY hawker_id ASC

```

Data Output Messages Notifications

Showing rows: 1 to 123 Page No: 1 of 1

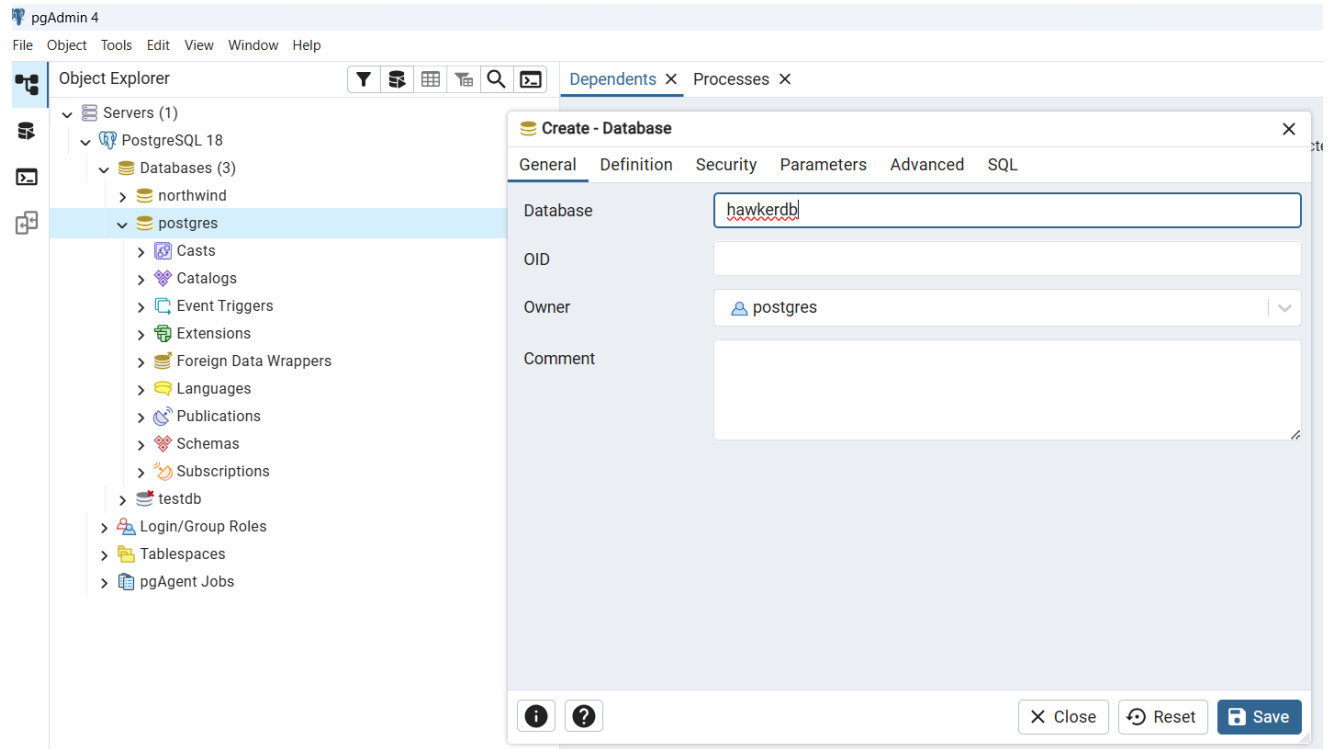
	hawker_id [PK] integer	name text	address text
1	1	Adam Road Food Centre	2, Adam Road, Singapore 289876
2	2	Aljunied Ave 2 Blk 117 (Blk 117 Aljunied Market and Food Centre)	Blk 117, Aljunied Ave 2, Singapore 380117
3	3	Amoy Street Food Centre (Telok Ayer Food Centre)	National Development Building, Annex B, Telok Ayer Street, Singapore 069...
4	4	Anchorvale Village Hawker Centre	339 Anchorvale Road, Singapore 540339
5	5	Ang Mo Kio Ave 1 Blk 226D (Kebun Baru Market and Food Centre)	Blk 226D, Ang Mo Kio Ave 1, Singapore 564226
6	6	Ang Mo Kio Ave 1 Blk 341 (Teck Ghee Court)	Blk 341, Ang Mo Kio Ave 1, Singapore 560341
7	7	Ang Mo Kio Ave 10 Blk 409 (Teck Ghee Square)	Blk 409, Ang Mo Kio Ave 10, Singapore 560409
8	8	Ang Mo Kio Ave 10 Blk 453A (Chong Boon Market and Food Centre)	Blk 453A, Ang Mo Kio Ave 10, Singapore 561453
Total rows: 123 Query complete 00:00:00.204			

CRLF Ln 2, Col 24

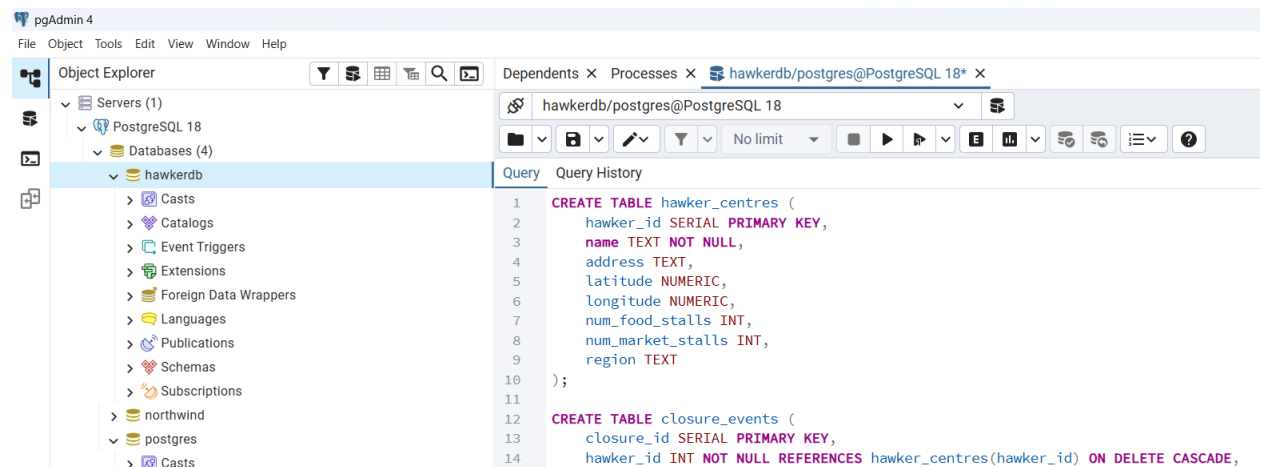
6. Database Schema and ERD

Screenshots:

- Creating the database



- Creating tables



Snapshot of source code used to create tables:

Jupyter PostgreSQL Table Creation Last Checkpoint: yesterday

File Edit View Run Kernel Settings Help Trust

JupyterLab Python [conda env:base] * ⌵

```
[ ]: # Run this PostgreSQL after creating hawkerdb database.

[ ]: CREATE TABLE hawker_centres (
    hawker_id SERIAL PRIMARY KEY,
    name TEXT NOT NULL,
    address TEXT,
    latitude NUMERIC,
    longitude NUMERIC,
    num_food_stalls INT,
    num_market_stalls INT,
    region TEXT
);

CREATE TABLE closure_events (
    closure_id SERIAL PRIMARY KEY,
    hawker_id INT NOT NULL REFERENCES hawker_centres(hawker_id) ON DELETE CASCADE,
    closure_type TEXT NOT NULL,
    start_date DATE NOT NULL,
    end_date DATE NOT NULL,
    remarks TEXT
);

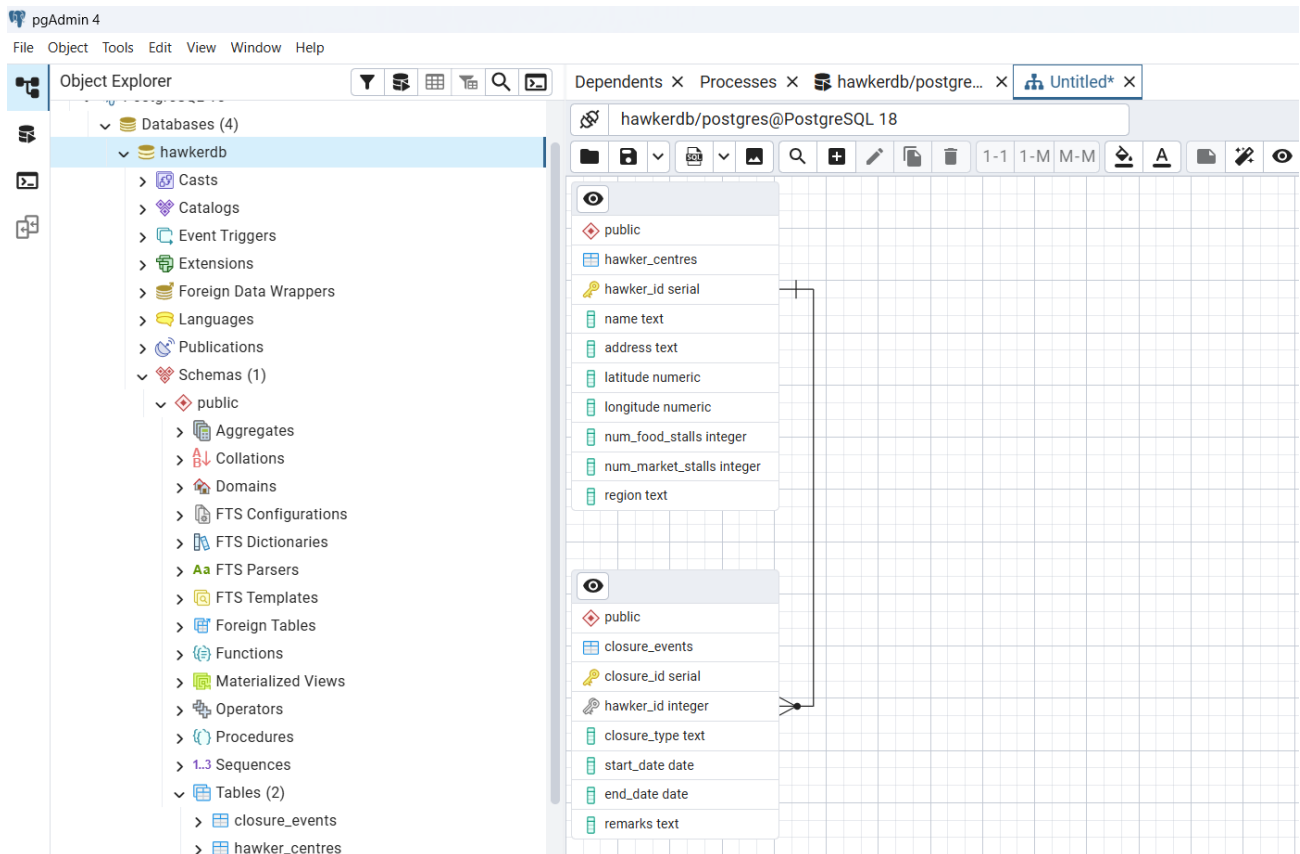
-- Optional indexes for analysis
CREATE INDEX idx_hc_region ON hawker_centres(region);
CREATE INDEX idx_ce_start_date ON closure_events(start_date);
CREATE INDEX idx_ce_hawker_id ON closure_events(hawker_id);
```

Tables

created:

⌵	🗃 Tables (2)
>	🗃 closure_events
>	🗃 hawker_centres

- ERD diagram



- Table structure for “closure_events”

closure_events

General

Columns

Advanced

Constraints

Partitions

Parameters

Security

SQL

Inherited from table(s)

Select to inherit from...

Columns

	Name	Data type	Length/Precision	Scale	Not NULL?	Primary key?	Default
	closure_id	integer					nextval(c
	hawker_id	integer					
	closure_type	text					
	start_date	date					
	end_date	date					
	remarks	text					

- Table structure for “hawker_centres”

closure_events

General

Columns

Advanced

Constraints

Partitions

Parameters

Security

SQL

Inherited from table(s)

Select to inherit from...

Columns

	Name	Data type	Length/Precision	Scale	Not NULL?	Primary key?	Default
 	closure_id	integer 					nextval('c
 	hawker_id	integer 					
 	closure_type	text 					
 	start_date	date 					
 	end_date	date 					
 	remarks	text 					

- Table Keys and Constraints

Table: “closure_events”

closure_events

General

Columns

Advanced

Constraints

Partitions

Parameters

Security

SQL

Primary Key Foreign Key Check Unique Exclude

+

	Name	Columns
 	closure_events_pkey	closure_id

Close

Reset

Save

closure_events

×

General

Columns

Advanced

Constraints

Partitions

Parameters

Security

SQL

Primary Key

Foreign Key

Check

Unique

Exclude

	Name	Columns	Referenced Table
✎	closure_events_hawker_id_fkey	(hawker_id) -> (hawker_id)	public.hawker_centres

?

?

Close

Reset

Save

- Table: “hawker_centres”

hawker_centres

×

General

Columns

Advanced

Constraints

Partitions

Parameters

Security

SQL

Primary Key

Foreign Key

Check

Unique

Exclude

	Name	Columns
✎	hawker_centres_pkey	hawker_id

?

?

Close

Reset

Save

hawker_centres

General

Columns

Advanced

Constraints

Partitions

Parameters

Security

SQL

Primary Key

Foreign Key

Check

Unique

Exclude

	Name	Columns	Referenced Table
--	------	---------	------------------

?

?

Close

Reset

Save

The “hawker_centres” table does not have a Foreign Key as it is the parent table. Only the **child table** that is the “closure_events” table holds the Foreign Key.

A relational schema is designed to organise closure information clearly and efficiently. The ERD includes core entities such as:

- **Hawker Centres** (centre details, region, location)
- **Closure Schedules** (start date, end date, duration, type of closure)

Primary and foreign keys are defined to maintain relationships between tables, enabling accurate joins and multi-table analysis.

7. Table Structure

Each table is created with explicit data types, constraints, and indexing where appropriate. Key design considerations include:

- Using DATE for closure periods
- Applying CHECK constraints for valid date ranges
- Using SERIAL or GENERATED keys for unique identifiers
- Ensuring region values follow a controlled vocabulary

This structure supports efficient querying and reduces the risk of data inconsistencies.

8. Data Validation

After loading the data into PostgreSQL, we verified that the database schema matched the intended design:

8.1 Schema Validation

- Correct table names (hawker_centres, closure_events)
- Correct column names and data types
- Primary keys and foreign keys were correctly applied
- No unexpected or missing columns

This ensures the database structure is consistent with the ERD.

8.2 Row count verification

We validated that the number of rows loaded into PostgreSQL matched the number of rows produced during the Transform stage.

That is:

- “hawker_centres” table row count matched the number of unique hawker centres in the dataset
- “closure_events” row count matched the number of valid closure records generated

This shows that no data was lost or duplicated during loading.

8.3 Data Type Validation

We checked that key fields were stored in the correct formats:

- Latitude/longitude converted to **numeric**
- Cleaning dates and other works dates converted to **DATE**
- Remarks stored as **text**
- hawker_id stored as **integer**

This ensures the data can be queried and analysed reliably.

8.4 Invalid or Missing Values

During transformation, we validated and cleaned the raw data:

- Converted invalid date strings (e.g., "NA", "NIL", "-") using errors="coerce"
- Dropped closure rows where start or end dates were missing

- Ensured hawker names mapped correctly to “hawker_id”
- Removed any rows where “hawker_id” could not be mapped

This step is necessary to prevent corrupted or incomplete records from entering the database.

8.5 Referential Integrity Checks

We validated the following:

- Every closure event has a valid “hawker_id”
- No orphaned closure records exist
- The foreign key relationship between “closure_events.hawker_id” and “hawker_centres.hawker_id” is consistent

This is done to ensure that the relational structure is accurate.

9. Sample Data Inspection

We ran the following queries in PostGreSQL to confirm that the ETL pipeline produced clean and accurate data.

Query Query History

```

1 SELECT * FROM hawker_centres LIMIT 10;
2 SELECT * FROM closure_events LIMIT 10;
3

```

Data Output Messages Notifications

	closure_id [PK] integer	hawker_id integer	closure_type text	start_date date	end_date date	remarks text
1	1	1	Cleaning	2026-03-30	2026-03-31	nil
2	2	1	Cleaning	2026-06-08	2026-06-08	nil
3	3	1	Cleaning	2026-09-07	2026-09-07	nil
4	4	1	Cleaning	2026-12-07	2026-12-08	nil
5	5	2	Cleaning	2026-03-09	2026-03-10	nil
6	6	2	Cleaning	2026-06-08	2026-06-09	nil
7	7	2	Cleaning	2026-09-14	2026-09-17	nil
8	8	2	Cleaning	2026-12-14	2026-12-15	nil
9	9	3	Cleaning	2026-02-28	2026-02-28	nil
Total rows: 10	Query complete 00:00:00.420					

After running the query above and observing the query results; we were able to confirm that the values, date formats, region assignments and mapping between hawker names and IDs are correct.

10. Analytical Framework

Once the data is validated, SQL queries were used to analyse:

- Closure frequency by region
- Duration patterns
- Monthly or seasonal trends
- Impact on food accessibility

This analytical layer forms the basis for insights and recommendations.

11. Challenges, Solutions and Limitations

11.1 Inconsistent Data Types (Latitude/Longitude as Strings)

Challenge: The API returned latitude and longitude values as **strings**, which caused type errors during region assignment and comparisons.

Solution: Converted coordinates to numeric using: `pd.to_numeric(..., errors="coerce")` This ensured all values were floats and allowed region logic to run correctly.

Limitation: Rows with invalid coordinates become NaN and cannot be assigned a region.

11.2 Invalid or Non-Standard Date Formats (“NA”, “NIL”, “-”)

Challenge: Closure date fields contained non-date strings such as "NA" and "NIL", causing “DateParseError” during “`pd.to_datetime()`”.

Solution: Used `errors="coerce"` to safely convert invalid dates to NaT, preventing pipeline failure.

Limitation: Some closure events may be dropped if both start and end dates are invalid, reducing completeness.

11.3 Name Mapping Issues (Hawker Name → `hawker_id`)

Challenge: Closure events must map to a valid “`hawker_id`”, but name mismatches or formatting differences can cause missing mappings.

Solution: Created a mapping dictionary from the loaded “`hawker_centres`” table and dropped rows where mapping failed.

Limitation: If the dataset contains inconsistent naming (for example; abbreviations), some closure events may be excluded.

11.4 API Pagination and Data Completeness

Challenge: The API returns data in pages and missing or incomplete pagination handling can lead to partial datasets.

Solution: Implemented a loop to fetch all pages until no more records were returned.

Limitation: If the API changes its pagination structure, the extraction logic will need to be updated.

11.5 Manual Region Assignment Logic

Challenge: Region classification was based on latitude/longitude thresholds, which may not perfectly match official boundaries.

Solution/Workaround: Implemented a simple rule-based function using coordinate cut-offs.

Limitation: This approach is approximate; official region boundaries may differ slightly.

12. Findings/Analysis

12.1 Problem statement 1:

Investigate if hawker centres' temporary closure schedules are equitably distributed across Singapore

12.1.1

Number of scheduled closures per region

disproportionately high number of closures (219), more than six times the number in the North and West (34 each). This indicates that closure schedules are not equitably distributed across regions.

12.1.2

Average closure duration per region

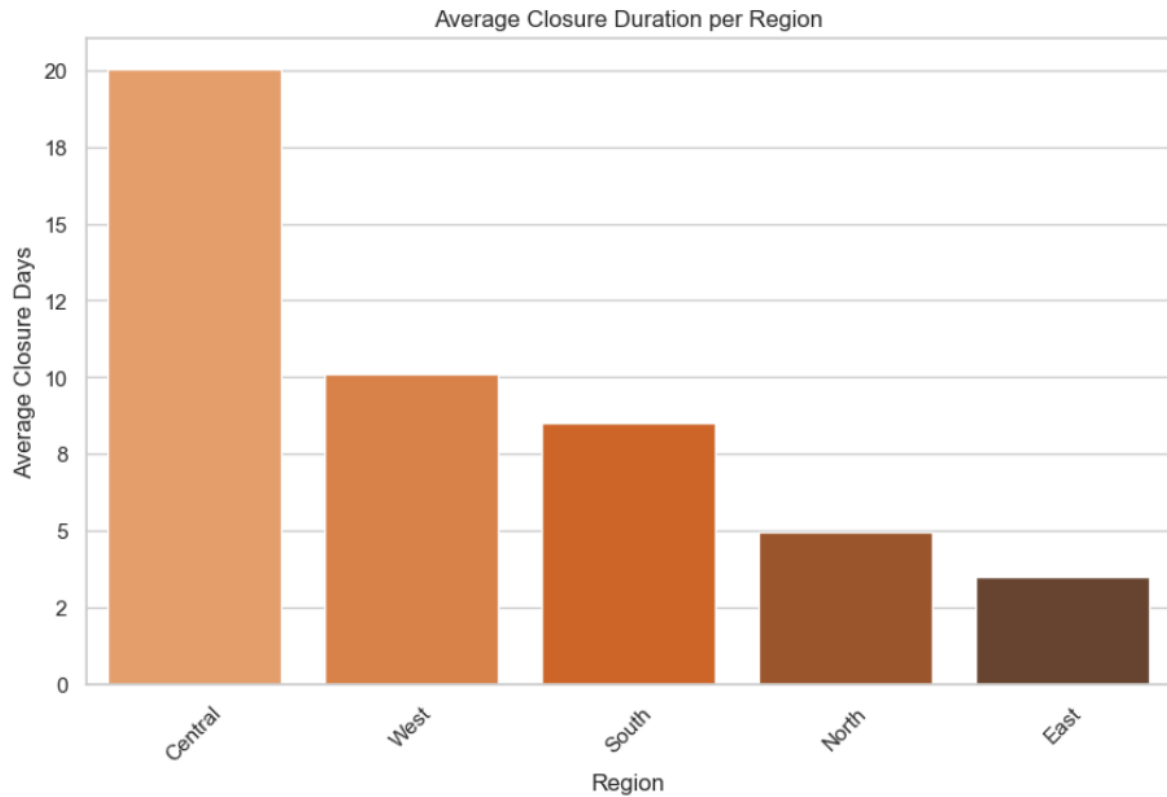
The screenshot shows a PostgreSQL query editor interface. The top bar indicates the connection is to 'hawkerdb/postgres@PostgreSQL 18'. Below the toolbar, the 'Query' tab is active, displaying the following SQL query:

```
1 SELECT hc.region,
2        AVG(ce.end_date - ce.start_date + 1) AS avg_closure_days
3 FROM hawker_centres hc
4 JOIN closure_events ce ON hc.hawker_id = ce.hawker_id
5 GROUP BY hc.region
6 ORDER BY avg_closure_days DESC;
7
```

Below the query editor, the 'Data Output' tab is active, showing the results of the query. The results are displayed in a table with two columns: 'region' (text) and 'avg_closure_days' (numeric). The table contains five rows, ordered by average closure duration in descending order.

	region	avg_closure_days
1	Central	20.0645161290322581
2	West	10.1176470588235294
3	South	8.5114155251141553
4	North	4.9411764705882353
5	East	3.4794520547945205

At the bottom of the interface, a status bar indicates 'Total rows: 5' and 'Query complete 00:00:00.123'.



The Central region experiences closure durations that are dramatically longer than all other regions, averaging 20 days per closure. This is nearly double compared to the West regions and more than six times the East region. This indicates a clear inequity in maintenance burden, with Central hawker centres facing disproportionately prolonged downtime.

12.1.3

Closures per hawker centre (hotspots)

hawkerdb/postgres@PostgreSQL 18

Query Query History

```

1 SELECT hc.name,
2       hc.region,
3       COUNT(*) AS num_closures
4 FROM   hawker_centres hc
5 JOIN   closure_events ce ON hc.hawker_id = ce.hawker_id
6 GROUP BY hc.name, hc.region
7 ORDER BY num_closures DESC;

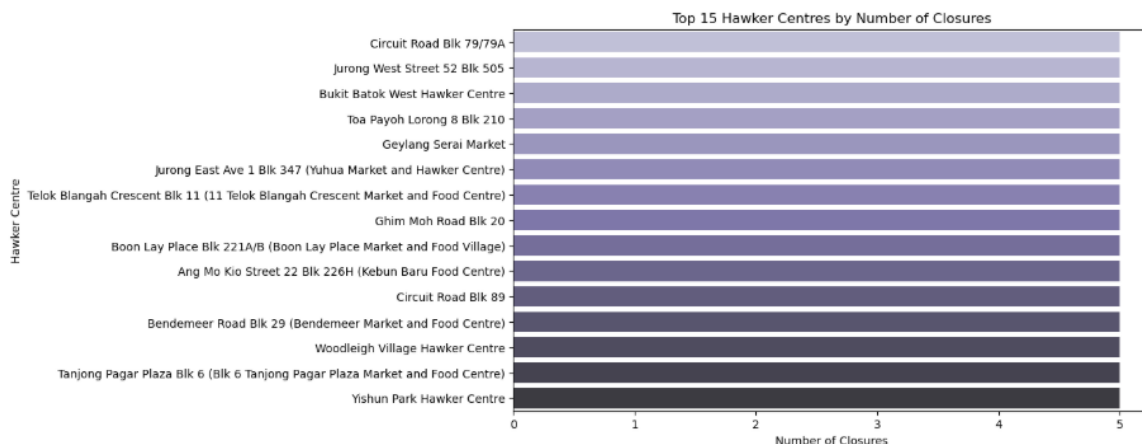
```

Data Output Messages Notifications

Showing rows: 1 to 123

	name text	region text	num_closures bigint
1	Circuit Road Blk 79/79A	Central	5
2	Jurong West Street 52 Blk 505	West	5
3	Bukit Batok West Hawker Centre	West	5
4	Toa Payoh Lorong 8 Blk 210	Central	5
5	Geylang Serai Market	South	5
6	Jurong East Ave 1 Blk 347 (Yuhua Market and Hawker Centre)	West	5
7	Telok Blangah Crescent Blk 11 (11 Telok Blangah Crescent Market and Food Ce...	South	5
8	Ghim Moh Road Blk 20	South	5
9	Boon Lay Place Blk 221A/B (Boon Lay Place Market and Food Village)	West	5

Total rows: 123 Query complete 00:00:00.207



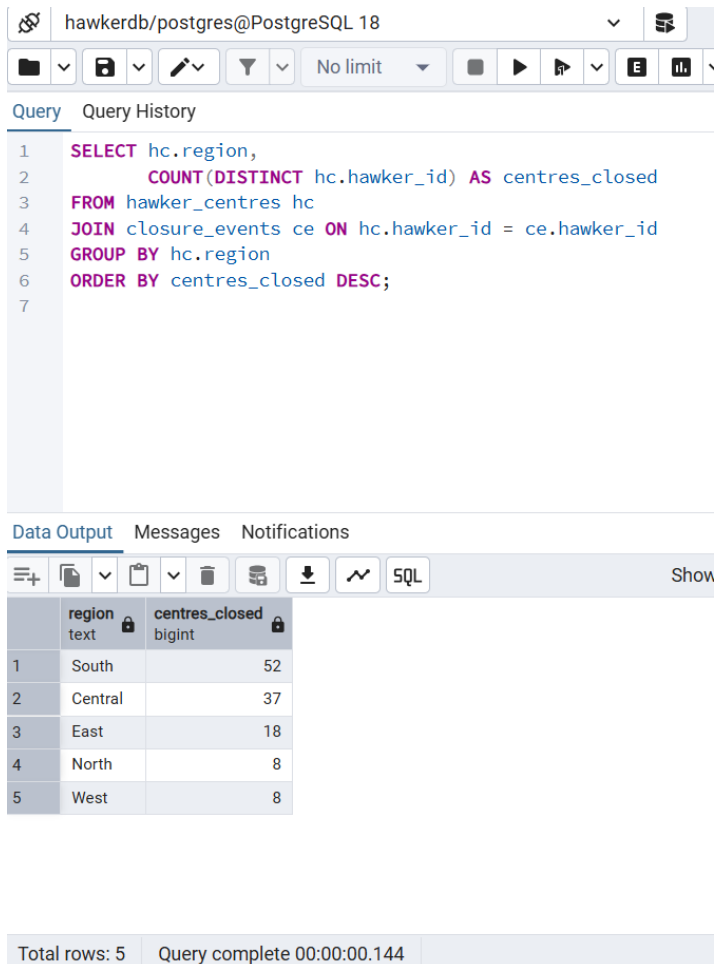
The chart above shows the top 15 hawker centres that experienced disproportionately high closure frequencies, indicating the presence of “maintenance hotspots.” These centres are most likely facing heavier operational stress, ageing infrastructure, or recurring repair needs, resulting in more frequent downtime compared to the national average.

12.2 Problem statement 2:

Assess the impact of hawker centre temporary closures on food accessibility and propose policy recommendations

12.2.1

Number of hawker centres closed per region



The screenshot shows a PostgreSQL query editor interface. The top bar indicates the connection is to 'hawkerdb/postgres@PostgreSQL 18'. Below the toolbar, the 'Query' tab is active, displaying the following SQL query:

```
1 SELECT hc.region,  
2        COUNT(DISTINCT hc.hawker_id) AS centres_closed  
3 FROM hawker_centres hc  
4 JOIN closure_events ce ON hc.hawker_id = ce.hawker_id  
5 GROUP BY hc.region  
6 ORDER BY centres_closed DESC;  
7
```

Below the query editor, the 'Data Output' tab is active, showing the results of the query in a table format. The table has two columns: 'region' (text) and 'centres_closed' (bigint). The results are as follows:

	region	centres_closed
1	South	52
2	Central	37
3	East	18
4	North	8
5	West	8

At the bottom of the interface, a status bar shows 'Total rows: 5' and 'Query complete 00:00:00.144'.

From this query result we can see that the South region experiences the highest number of hawker centres affected by closures that is; 52, far exceeding all other regions. This indicates that closure events in the South affects a significantly larger number of physical hawker locations.

12.2.2

Identify months with highest closures (food access pressure)

hawkerdb/postgres@PostgreSQL 18

No limit

Query Query History

1

2

3

4

5

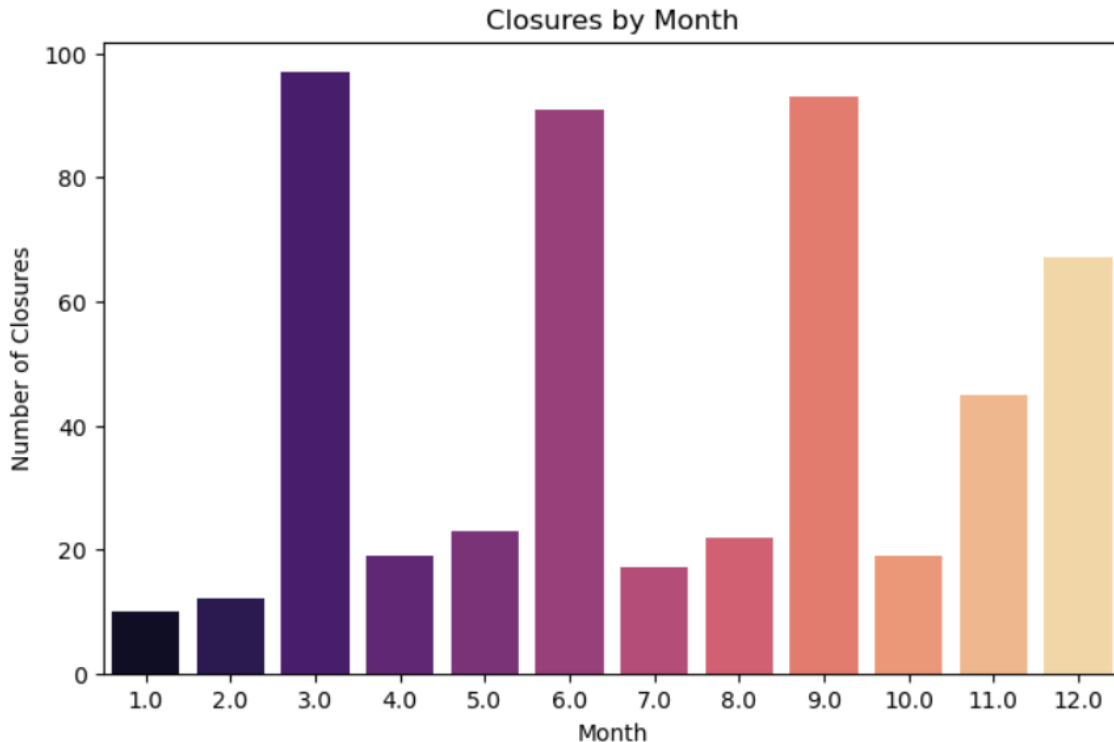
SELECT EXTRACT(MONTH FROM start_date) AS month,
COUNT(*) AS num_closures
FROM closure_events
GROUP BY month
ORDER BY num_closures DESC;

Data Output Messages Notifications

SQL

	month numeric	num_closures bigint
1	3	97
2	9	93
3	6	91
4	12	67
5	11	45
6	5	23
7	8	22
8	4	19
9	10	19

Total rows: 12Query complete 00:00:00.098



From the query result and chart above we can derive that closures peak sharply in **March, September** and **June**. This indicates clear seasonal patterns in maintenance cycles. These peak months create **concentrated food accessibility pressure**, as significantly more hawker centres undergo downtime simultaneously compared to the rest of the year.

13. Next Steps/Conclusion

Combined conclusion for problem statements 1 and 2

The analysis across both problem statements demonstrates that hawker centre temporary closures in Singapore are not equitably distributed across regions or time. Central and South experience significantly higher closure frequencies, longer closure durations and a larger number of affected hawker centres and food stalls. These disparities translates directly into unequal food accessibility pressures, with the South facing the greatest disruption and peak months such as March, September, and June intensifying the impact. Overall, the findings show that both the operational burden of closures and their consequences for residents and stallholders are unevenly distributed, highlighting the need for more balanced, data-driven closure planning.

14. Recommendations (Problem Statement 1 & 2 Combined)

14.1

Implement Region-Balanced Closure Scheduling

To address inequity in closure distribution, NEA should ensure that no single region particularly the Central and South regions, should not experience disproportionately frequent or prolonged closures. A region-balanced scheduled maintenance framework will distribute maintenance more evenly across Singapore.

14.2

Stagger Closures Within High-Impact Regions

In regions with many hawker centres (e.g., South and Central), closures should be staggered so that multiple large centres do not close at the same time. This reduces cumulative disruption to residents and prevents sudden drops in food availability.

14.3

Limit Major Closures During Peak Months

Since closures peak in March, September, and June, NEA should cap the number of major closures allowed during these months. This prevents seasonal spikes in food accessibility pressure and spreads maintenance more evenly across the year.

14.4

Prioritise Upgrading and Preventive Maintenance in High-Burden Regions

Hawker centres in Central experience the longest closure durations, indicating deeper structural issues. Prioritising these centres for upgrading, repairs, and predictive maintenance can reduce the need for long downtimes in the future.

14.5

Provide Temporary Support for Stallholders During Long Closures

To reduce income loss for hawkers affected by prolonged or repeated closures, NEA could consider:

- rental rebates
- temporary relocation options
- short-term financial assistance
- digital ordering pop-ups or temporary booths

This supports livelihoods while maintaining food availability.

14.6

Deploy Temporary Food Solutions in Areas With Multiple Closures

In high-impact regions like the South, temporary food kiosks, mobile food trucks, or pop-up cooked-food booths can help maintain food access when several centres are closed simultaneously.

14.7

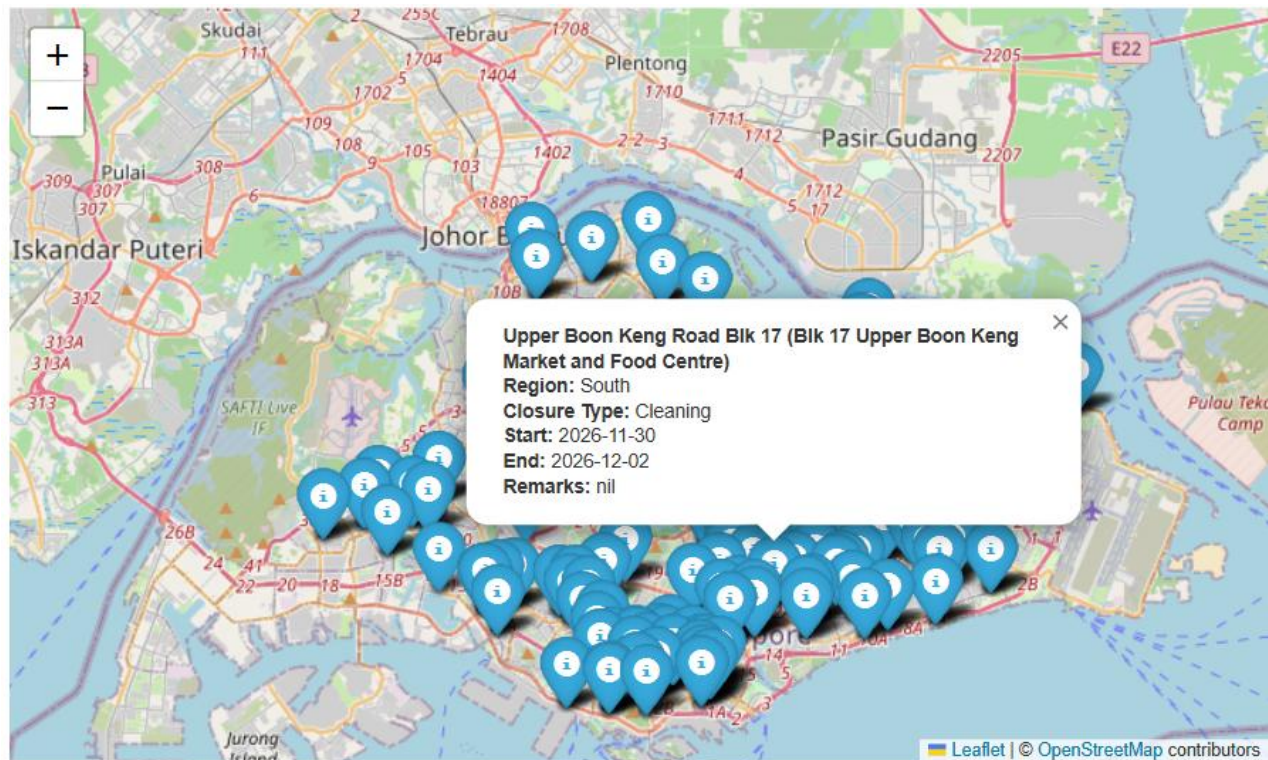
Use Data-Driven Equity Assessments for Future Closure Planning

NEA should incorporate metrics such as:

- number of stalls affected
- closure duration
- population density
- regional dependency on hawker centres

This could help in ensuring that hawker centre closures do not have a significant impact on residents.

15. Pop Up Map showing hawker centre closures



This interactive pop-up map shows the hawker centres that are currently closed for cleaning and/or maintenance work. This will be demonstrated in our presentation.